

15-112 Exam 2 Review

Up to 40 minutes. No calculators, no notes, no books, no computers. Show your work!
Do not use try/except on this review.

1. (points) CT2

```
import copy

def ct2(L):
    M = copy.copy(L)
    N = copy.deepcopy(L)
    O = L[:]
    M[0].append(3)
    N[0].append(4)
    O += [3]
    L.insert(0, L.pop())
    print(f"M: {M}")
    print(f"N: {N}")
    print(f"O: {O}")

L = [[1], [2]]
ct2(L)
print(f"L: {L}")
```

2. (points) CT2

```
import copy

def ct2(L, A, n):
    A[-1][0] += n
    A[n%2] += [10*n]
    L.append(n)
    print(f'A: {A}')

L = [[3], [4]]
C = copy.copy(L)
D = copy.deepcopy(L)
ct2(L, C, 1)
ct2(L, D, 2)
print(f'L: {L}')
```

3. (points) **Free Response:** Sub lists

Write the function `isSubList(L1, L2)` which, given two lists, returns `True` if `L1` is a sub list of `L2` and `False` otherwise.

`L1` is a sub list of `L2` if all of its elements are next to each other and in the same order inside of `L2`. For example, `[6, 3, 8]` is a sub list of `[1, 6, 3, 8, 5]` because the elements `6, 3, 8` can be found together, and in the same order, in the middle of `[1, 6, 3, 8, 5]`. However, `[6, 3, 8]` is *not* a sub list of `[6, 1, 3, 8, 5]` because even though all of the elements are there in order, they are not next to each other.

Consider the following test cases:

```
# Normal Cases
```

```
assert isSubList([6, 3, 8], [1, 6, 3, 8, 5]) == True
assert isSubList([6, 3, 8], [1, 8, 3, 6, 5]) == False
assert isSubList([6, 3, 8], [6, 1, 3, 8, 5]) == False
assert isSubList([6, 3, 8], [6, 3, 8]) == True
assert isSubList(["cat", "dog", "horse"], ["mouse", "monkey", "cat", "dog", "horse"]) == True
assert isSubList([10, 20], [50, 40, 8, 10, 7]) == False
```

```
# Strange case
```

```
assert isSubList([], [2, 1, 3, 4, 5]) == True
```

Hint: Slicing out pieces of `L2` to compare to `L1` can be helpful here.

4. (points) **Free Response:** Animations: Stop the Circle!

Write a game animation with the following features.

1. When the game starts, a red square with side lengths of 50 pixels is drawn centered on the canvas, and a blue circle of radius 20 pixels is drawn at a random location on the canvas.
2. When the user clicks inside the circle immediately after clicking inside the red square, the circle radius decreases by half, and the circle moves to a new random location.
3. Four times per second, the circle's radius grows by 1 pixel.
4. If any part of the circle ever goes beyond the edge of the canvas, a game-over message appears at the center of the canvas, and the square and the circle are removed from the screen.
5. If the game is over, the user can restart the game by pressing r.

Notes:

- Do not hardcode for a 400x400 canvas. However, you may assume the canvas is square and at least 100x100 pixels.
- You will be penalized if your code results in an MVC violation.
- Make reasonable choices for anything not specified above.
- To solve this, you need to write `onAppStart`, `onKeyPress`, `onMousePress`, `redrawAll`, and `onStep`.
- You do not need to include any imports or the main function.